



Obsolescence... Could it soon be a thing of the past?

This month **Edge** got a glimpse of the future, thanks to a demonstration of the **Tos** operating system. Full details can be found on page 10, but in a nutshell **Tos** enables programs coded on any machine to run on any other machine – in parallel, across any available processors in the system.

And because of the way **Tos** programs are structured, the system could open the way to a computer that can be upgraded simply by plugging in extra processors; **Tos** recognises the new chips and allocates components of the code accordingly.

So, for instance, instead of buying a new game, you could spend your money on an upgrade pack to speed up the games you already have. With the beneficial side effect that your console is safeguarded against obsolescence.

This ingenious software could have huge ramifications for the entire computer and videogame industry – if it becomes an accepted operating system. But given that **Tos** Systems – the company behind it – are already talking to some major Japanese corporations and have close links with luminaries in Bristol, this is a real possibility.

Tos is even more amazing when you realise that it is the product of one man's efforts, coding for his own benefit, rather than the cumulative efforts of some corporate programming team.

In that respect, **Tos** is an echo of the pioneering days of computer games, when lone programmers working at home managed to create astonishingly compact and efficient code.

And with a programming tool like **Tos** at their disposal, those groundbreaking programmers could have their day once more...

The future is almost here...

Taos: setting new standards

Edge examines a new operating system that could change the world



Three wise men: (from left to right) Chris Hinesley (inventor of Taos), Tim Moore and Francis Chung - the directors of Tao Systems

The constant evolution of videogames hardware has meant that systems have been dogged by the perennial problems of disparate architectures, splintered markets, obsolescence and lack of portability. But all that could change with the advent of Taos (pronounced *downs*) - a global, heterogeneous, parallel-processing operating system with no speed overruns and a minuscule kernel size.

Taos is the brainchild of Chris Hinesley - programmer of such classics as *Polarisania*, *Everone's A Nally*, *Rockbyte* and *Orinlight*.

"I started in the days when the Z801 came out, and moved up through most of the machines that have come along since then," says Chris. "When the move to 386 machines came around, it became obvious that the porting times from one architecture to another was becoming prohibitive - it was taking more and more time doing something that you'd already written once, and you then had to write it all again for another machine."

Chris had the idea of producing an operating system that would manage

games and also aid code portability. The first step was a macro set which Chris constructed for the assemblers of all the platforms he was writing on. Rather than write in the native assembler language, he wrote in the macro language he'd defined; he then devised a translator which would take a binary equivalent of that macro set and translate it, on the fly, into the instructions for a particular machine.

"This 'virtual processor' works like a 16-bit register RISC microprocessor," explains Chris. "But it isn't an emulated technology: it actually translates into native code and it's the native code which runs and all the translations take place during the load time of the code."

When developing the system, Chris realised that standard libraries would be needed - particularly on the GUI (Graphical User Interface) side, such as window managers, graphic drawing tools, polygon generators, MPES video and so on. It was then that he drew on the talents of **Tim Moore**.

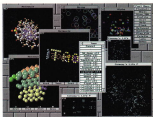
The two first met when Chris was working for Microgen and Tim was doing Amstrad conversions for Digital →

By definition

Tao (that in virtue of which all things happen or exist); the rational basis of human conduct; the course of life and its relation to eternal truth. (Chinese, literally: path, way)



Taos comes with its own graphical interface which boasts fully resizable windows and is easy to use. Unlike some window programs...



Taos' versatility is shown in its ability to multitask a number of programs in parallel - and even the same program with different tasks.

Taos features

Hardware independence: Taos applications run on different processor architectures without having to recompile or reconfigure the code.
Load balancing: an optimum distribution of processes over the processor network.
Heterogeneous processing: Parallel applications are able to operate over networks of dissimilar processors.
Dynamic binding: Only those parts of an application which are needed at any time are loaded into memory.
Multi-threading: A piece of code loaded into memory is available to all programs that need it.
Parallelism: Taos uses a process-based programming model, allowing parallel programs to be written more easily.
Minimal kernel: Taos' basic code is very compact - about 100K. This optimises performance and minimises memory requirements.

+ **Integration.** Their first product together was the Atari ST version of Verminator for Rainbird - at which point Teleconsoft hit financial difficulties and was bought out by Microprose.

"Tim was working on a raytracer for the Amiga and he'd done some work on the ST," recalls Chris, "and that's when we had the idea of doing a parallel system - the idea of being able to plug in more processors to make the system run faster. At first it was simply a desire to speed up Tim's tracer."

At this

time - the mid-'80s - transputers were all the rage. The guiding concept behind transputers was that they could run in parallel: the more chips you plugged into a board, the faster your application ran.

Tim initially set about attempting to generate workstation performance on a PC. "We had the idea of adding T800 transputers to improve the performance of that system," states Tim. "I don't think we'd considered doing different microprocessor architectures on the same system, but because we had this binary portability, it meant it wasn't as difficult as other people were finding."

Chris agrees: "I coded up a small kernel for the transputer system and that was ported from the 68000 version of the system. And it became obvious very quickly that in doing this I'd done something that everybody else - research institutes and universities - had been trying to do for a long time."

Seeing the potential of the system, the duo decided to turn it into a proper operating system and market it as such, and the result was Taos. When

the venture capital company funding them collapsed, **Francis Charig** stepped in to save the day. Francis had experience in the computer business, had worked in the stock exchange and had extensive Japanese contacts which would later prove invaluable.

"Francis came from a very good business background and he could see the potential of the product. It's really Francis that has driven the marketing since then, leaving me and Tim to work on the technical side," states Chris.

When talking about Taos, the usually quiet-spoken Francis becomes even more animated and enthusiastic than the system's inventors: "There are already various blue-chip companies - especially in Japan, where our funding comes from - who believe Taos is an operating environment way ahead of any other operating environment in the world - significantly way ahead of Unix or DOS or anything like that."

In fact, Taos has been tested around several very large Japanese companies both as a dedicated operating system and - with its library support for MPEG, PostScript, and real-time polygon rendering - as the basis for a set-top box or games console.

Francis goes on to explain how Taos' parallelism means that the system can be expanded, apparently without limit, providing vast power capabilities. And it's also



Chris built a console prototype which utilises transputer 'cards' like these you plug in the faster it runs.

heterogeneous - different types of processors can be plugged together to form, in effect, one system. "You can get a 486 and an SGI MIPS chip and plug them together and with Taos you can run a parallel raytracer like Tim's raytracer across the 486 and the MIPS processor and it's distribute it in an optimum fashion across the network," enthuses Francis.

Chris adds: "The system automatically load-balances across →

What is it?

It's a 16bit machine, but can display 4,096 colours onscreen, has 385 sprites, three planes of scrolling and robust hardware scaling. It can run games as large as 130 megabits (around 48Mb), and the carts can cost as much as £220.

BUZZ words

net surfin'

being away your ski boots dude surfin' where it's at everybody's goin' surfin' like in that song hey you can hang ten shoot the tube ride some glassy barrels on these billions of bulletin board things with your sad pc makes no need to go out again dude just get your pc wired up and off you go for an orgy of surfin' in silicon valley man can billions of thoughts and ones for free next i'm gonna start net swimmin' man some sort of thing except your thoughts and ones get wetter i sposed there net cookin' loads of new recipes thing is i'm shibe at cooking... hrm net steakin' roach net shaggin' car peah...

it is...

SNK's Neo-Geo console and arcade system. First seen in 1990 it represented the state of the art in videogaming – and still manages to turn heads today. We have the full lowdown on this powerful kit in Edge 10

the network for you, and when you add a new processor, it acknowledges the fact that there's a new processor on the network. If you're running a sufficient number of sub-processors on your application it will take those processors and will distribute them across the additional processors.

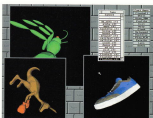
'There is no requirement to recompile the program or anything along those lines; it's all handled automatically. That's what has been holding back parallel processing generally, which is why Taro is being addressed by a lot of companies, particularly in London and Japan, as a generic operating system.'

Taro's ability to address several processors in parallel has led to the idea of upgradeable games hardware. As the technology improves, you simply plug in an extra processor pack, and Taro will automatically make use of the extra power, running programs with improved efficiency and speed.

'The sort of thing that we're talking to people about is an open console into which you can plug modules which can be of a dissimilar type,' says Francis. 'If you're a games manufacturer producing consoles today, in two years' time do you really want to be using the same architecture? And you're not really making your money from the consoles, but from the software; the consoles are simply an enabler for revenue. If they can provide an open-ended environment which allows consumers to just continue plugging in module after module, it's a great benefit.'

Apart from

its obvious benefits to the videogames community, Taro could also be instrumental in



Taro already has the support of thousands of programming tools – such as the 3D polygon generator displaying its wares here

bringing game development back to the amateurs who started it all. Chris points out: 'In the old days you had this influx of kids who could buy a Spectrum, could start programming, and then (could by their hand at doing arcade games. That was the lifeblood of the industry, and that's been gradually draining away because I don't know many kids who can afford thousands of pounds for a console development system. But they can certainly afford a PC; they can develop on the PC, and Taro will allow it to run on the ARM, on the new super-duper consoles from everybody else.'

'You can actually develop on your PC for an arcade system, without any problem,' adds Francis. 'In fact, we're doing that in-house right now. We have custom technology that we're developing for the arcade market as a slightly separate issue from the approach to licensing for console manufacturers. That will give people the ability, if they have a good game that's reasonably successful on the console market, to turn it into an arcade machine just by saying they want to turn it into an arcade machine. And, of course, the reverse step-up in going from PCs and consoles into the arcade market must be attractive for games developers...'

So how

exactly does this revolutionary system achieve such impressive results? Taro works by emulating a virtual processor (VP) – an Imaginix 16/32bit-processor machine. All compiled code is written in the language invented for the VP and targeted at this virtual processor. →



With settings boxes in mind, Taro has support for a software MPEG player (inset, bottom left) and Postscript graphics (on the right)

+ rather than the native processor in the target system.

The Taos kernel (the central code), which is typically around 50K, is loaded into the processor at boot time and executes. That kernel is specific to whatever CPU it's running on, although each kernel is written in the virtual processor's code.

If this kernel finds it needs a translator tool, it brings in the translator as well. The application then gradually builds itself in memory: as a processor in the network needs to call functions it brings them in and loads the application.

All programs are compiled or assembled into VP code and are kept in this form on disk. The VP code is translated into the native code of the processor on which it is to run only when it is needed. The translation occurs as the VP code is loaded from disk, across the network, and into the memory of the target processor.

However, this doesn't slow the system down: most processors can actually translate VP code into native code faster than VP code can be loaded from disk and sent across the network. And VP code is often more compact than native code: it takes up less disk space and is loaded faster.

For instance, if you had a console that loaded from CD-ROM, a CD would be pressed so that the first thing it did would be to load up the appropriate version of Taos, place it in memory and set it running. Then it would load the game code, which would run under the operating system. The operating system would then load the specific tools required for that game and execution of the game would begin.

Access to custom chips is taken care of automatically by Taos using a



From Pyramus to global operating system, Chris Hensley: inventor of Taos and bloody clever Mike

method called dynamic binding: individual chips are supported by VP libraries, which allow a tool for that particular processor to be accessed by the system; the tools are loaded in during runtime as they are needed. Dynamic binding also enables several processes to share tools, which is very memory efficient.

Conventional operating systems contain a library of around 200 tools, and the entire library is loaded even if you just want to call one of the tools. Taos has libraries down to the one function level: when a specific function is needed, it gets loaded in on its own. The libraries or components of the operating system not wanted simply don't exist as far as the application is concerned, because they never get called into memory. This is good for games programmers because it enables them to pick and choose what functions to use; they can dispense with a particular part of the operating system and use their own code instead, without any conflicts arising.

Taos is undoubtedly a radical advance which is deservedly attracting interest from the giants of the videogames industry. The principles of the system may be opaque to non-programmers, but if it fulfils its promise of programs which can run on any system and be able to call on limitless processing power, Taos could soon be a truly universal tool. And it's nice to see something that addresses global problems rather than those of a single system.



Data stream

Total number of computer and videogame outlets in the UK: **4,900**
Number of outlets owned by a major chain: **2,750**
Average price of a game on floppy disk: **£20**
Average price of a game on cartridge: **£45**
Biggest-selling game of 1993 on a single format: **FIFA Soccer (Mega Drive)**
Biggest-selling multi-format game of 1993: **Mortal Kombat**
Number of software publishers currently signed up to develop Jaguar software: **78**
Number of games known to be in development for the Jaguar: **44**
Jag's share of the software market in 1993: **39.4%**
Nintendo's share of the software market in 1993: **33.3%**
Percentage of the total software market taken by Mega Drive games in 1993: **32.7%**
Percentage of the software market taken by 1993 games in 1993: **18.5%**
Percentage of the total software market taken by Mega CD games in 1993: **1**
Biggest-selling disc-based game of 1993: **Plantar: One 2 (Amiga)**
Net revenues for Silicon Graphics' fourth quarter, 1993: **\$378 million**
Increase compared to same period, 1992: **37%**
Number of Challenger tank simulators ordered from Silicon Graphics by users in the UK: **82**
Cost of one Challenger simulator: **\$40,000**
Number of PC compatibles in the US with CD-ROM drives at end of 1993: **7 million**
Number of Apple Macintoshes in the US with CD-ROM drives at end of 1993: **1.7**

Tao Systems: 082 935 5706



MPEG video, 3D polygon generator and Tim Mower's raytracer all running under Taos. And we haven't heard the best of that raytracer...